

# TEAM 4

## 2026 Security Assessment Report Prepared For



Report Issued: April 2026



# TABLE OF CONTENTS

|                                         |    |
|-----------------------------------------|----|
| TABLE OF CONTENTS                       | 2  |
| Confidentiality Notice                  | 3  |
| Disclaimer                              | 3  |
| EXECUTIVE SUMMARY                       | 3  |
| Recommendation                          | 4  |
| HIGH LEVEL ASSESSMENT OVERVIEW          | 5  |
| Observed Security Strengths             | 5  |
| Areas for Improvement                   | 5  |
| SCOPE                                   | 6  |
| Networks                                | 6  |
| TESTING METHODOLOGY                     | 7  |
| CLASSIFICATION DEFINITIONS              | 8  |
| Risk Classifications                    | 8  |
| Exploitation Likelihood Classifications | 8  |
| Business Impact Classifications         | 9  |
| Remediation Difficulty Classifications  | 9  |
| ASSESSMENT FINDINGS                     | 10 |
| TOOLS USED                              | 27 |
| ENGAGEMENT INFORMATION                  | 28 |
| Client Information                      | 28 |
| Version Information                     | 28 |
| Contact Information                     | 28 |

---

## Confidentiality Notice

This report contains sensitive, privileged, and confidential information. Precautions should be taken to protect the confidentiality of the information in this document. Publication of this report may cause reputational damage to OWASP Juice Shop or facilitate attacks against OWASP Juice Shop. Team 4 shall not be held liable for special, incidental, collateral or consequential damages arising out of the use of this information.

## Disclaimer

*Note that this assessment may not disclose all vulnerabilities that are present on the systems within the scope of the engagement. This report is a summary of the findings from a “point-in-time” assessment made OWASP Juice Shop’s environment. Any changes made to the environment during the period of testing may affect the results of the assessment.*

## EXECUTIVE SUMMARY

Team 4 performed a security assessment of the internal corporate network of OWASP Juice Shop on April 2026. Team 4's penetration test simulated an attack from an external threat actor attempting to gain access to systems within the OWASP Juice Shop corporate network. The purpose of this assessment was to discover and identify vulnerabilities in OWASP Juice Shop's infrastructure and suggest methods to remediate the vulnerabilities. Team 4 identified a total of 15 vulnerabilities within the scope of the engagement which are broken down by severity in the table below.

| CRITICAL | High | MEDIUM | LOW |
|----------|------|--------|-----|
| 1        | 4    | 7      | 3   |

The highest severity vulnerabilities give potential attackers the opportunity to SQL Injection enabling full administrative account takeover, stored/reflected/DOM-based XSS vulnerabilities, unauthenticated access to sensitive FTP files, and broken access control allowing horizontal privilege escalation into other users' shopping baskets. In order to ensure data confidentiality, integrity, and availability, security remediations should be implemented as described in the security assessment findings.

Note that this assessment may not disclose all vulnerabilities that are present on the systems within the scope. Any changes made to the environment during the period of testing may affect the results of the assessment.

## Recommendation

This is an optional paragraph that discusses a very critical series of business failures (e.g. failure to adhere to applicable legal regulations) that isn't a technical vulnerability but still should be brought to the attention of the executive team.



# HIGH LEVEL ASSESSMENT OVERVIEW

## Observed Security Strengths

Team 4 identified the following strengths in OWASP Juice Shop's network which greatly increases the security of the network. OWASP Juice Shop should continue to monitor these controls to ensure they remain effective.

## Areas for Improvement

Team 4 recommends OWASP Juice Shop takes the following actions to improve the security of the network. Implementing these recommendations will reduce the likelihood that an attacker will be able to successfully attack OWASP Juice Shop's information systems and/or reduce the impact of a successful attack.

---

## SCOPE

All testing was based on the scope as defined in the Request For Proposal (RFP) and official written communications. The items in scope are listed below.

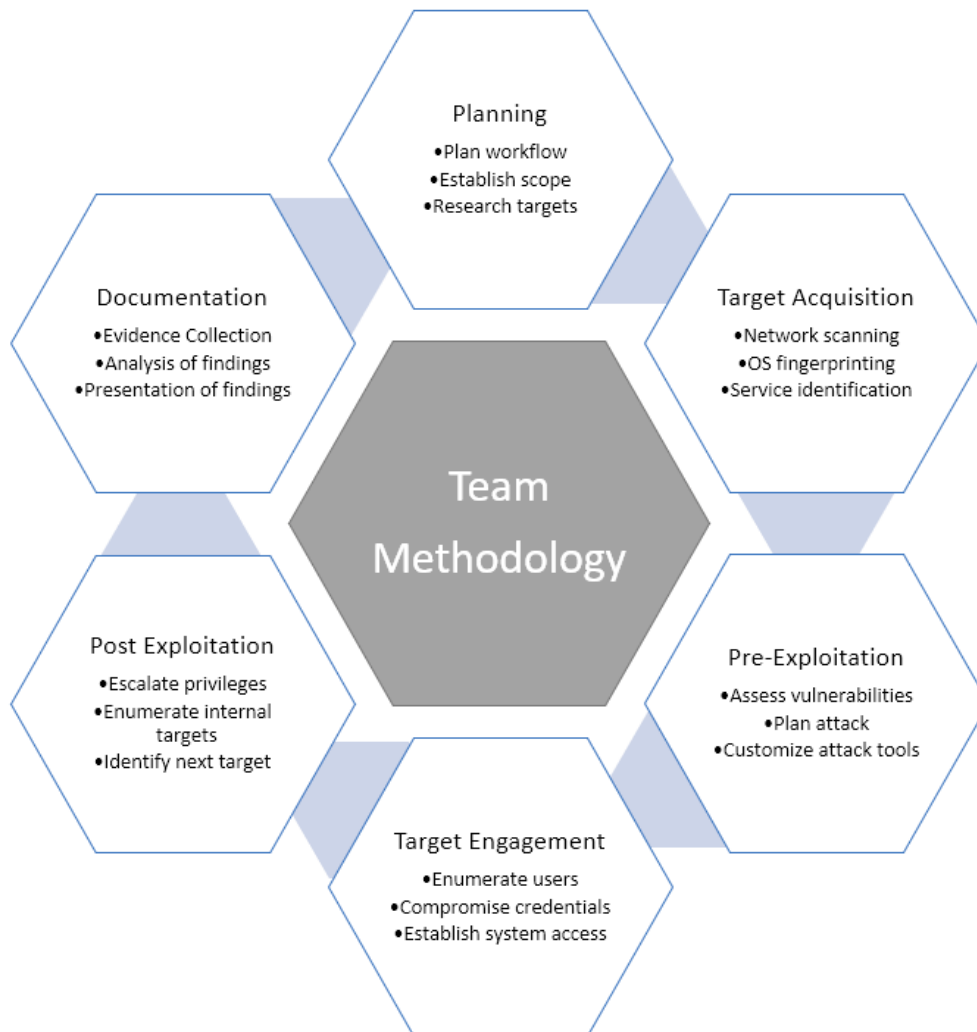
## Networks

| Network          | Note                                                     |
|------------------|----------------------------------------------------------|
| 10.114.140.87/32 | OWASP Juice Shop web application host<br>(TryHackMe Lab) |

# TESTING METHODOLOGY

Team 4's testing methodology was split into three phases: *Reconnaissance*, *Target Assessment*, and *Execution of Vulnerabilities*. During reconnaissance, we gathered information about OWASP Juice Shop's network systems. Team 4 used port scanning and other enumeration methods to refine target information and assess target values. Next, we conducted our targeted assessment. Team 4 simulated an attacker exploiting vulnerabilities in the OWASP Juice Shop network. Team 4 gathered evidence of vulnerabilities during this phase of the engagement while conducting the simulation in a manner that would not disrupt normal business operations.

The following image is a graphical representation of this methodology.



# CLASSIFICATION DEFINITIONS

## Risk Classifications

| Level                | Score      | Description                                                                                                                                                                       |
|----------------------|------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Critical</b>      | <b>10</b>  | The vulnerability poses an immediate threat to the organization. Successful exploitation may permanently affect the organization. Remediation should be immediately performed.    |
| <b>High</b>          | <b>7-9</b> | The vulnerability poses an urgent threat to the organization, and remediation should be prioritized.                                                                              |
| <b>Medium</b>        | <b>4-6</b> | Successful exploitation is possible and may result in notable disruption of business functionality. This vulnerability should be remediated when feasible.                        |
| <b>Low</b>           | <b>1-3</b> | The vulnerability poses a negligible/minimal threat to the organization. The presence of this vulnerability should be noted and remediated if possible.                           |
| <b>Informational</b> | <b>0</b>   | These findings have no clear threat to the organization, but may cause business processes to function differently than desired or reveal sensitive information about the company. |

## Exploitation Likelihood Classifications

| Likelihood      | Description                                                                                                                                                                                              |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Likely</b>   | Exploitation methods are well-known and can be performed using publicly available tools. Low-skilled attackers and automated tools could successfully exploit the vulnerability with minimal difficulty. |
| <b>Possible</b> | Exploitation methods are well-known, may be performed using public tools, but require configuration. Understanding of the underlying system is required for successful exploitation.                     |
| <b>Unlikely</b> | Exploitation requires deep understanding of the underlying systems or advanced technical skills. Precise conditions may be                                                                               |





|  |                                       |
|--|---------------------------------------|
|  | required for successful exploitation. |
|--|---------------------------------------|

## Business Impact Classifications

| Impact          | Description                                                                                                                                      |
|-----------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Major</b>    | Successful exploitation may result in large disruptions of critical business functions across the organization and significant financial damage. |
| <b>Moderate</b> | Successful exploitation may cause significant disruptions to non-critical business functions.                                                    |
| <b>Minor</b>    | Successful exploitation may affect few users, without causing much disruption to routine business functions.                                     |

## Remediation Difficulty Classifications

| Difficulty      | Description                                                                                                                                                      |
|-----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Hard</b>     | Remediation may require extensive reconfiguration of underlying systems that is time consuming. Remediation may require disruption of normal business functions. |
| <b>Moderate</b> | Remediation may require minor reconfigurations or additions that may be time-intensive or expensive.                                                             |
| <b>Easy</b>     | Remediation can be accomplished in a short amount of time, with little difficulty.                                                                               |

## ASSESSMENT FINDINGS

| Number | Finding                                                      | Risk Score | Risk     | Page |
|--------|--------------------------------------------------------------|------------|----------|------|
| 1      | SQL Injection - Admin Login Bypass                           | 10         | Critical | 13   |
| 2      | SQL Injection - Account Takeover (Bender)                    | 8          | High     | 14   |
| 3      | Broken Authentication - Weak Admin Password                  | 8          | High     | 15   |
| 4      | Persistent XSS via HTTP Header (True-Client-IP)              | 8          | High     | 16   |
| 5      | Broken Access Control - Admin Panel Exposed                  | 7          | High     | 17   |
| 6      | Reflected XSS via Order Tracking ID                          | 6          | Medium   | 18   |
| 7      | DOM XSS via Search Bar                                       | 6          | Medium   | 19   |
| 8      | IDOR - View Other Users' Baskets                             | 6          | Medium   | 20   |
| 9      | Sensitive Data Exposure - Public /ftp/ Directory             | 6          | Medium   | 21   |
| 10     | Broken Authentication - Password Reset via Security Question | 5          | Medium   | 22   |
| 11     | Broken Authentication - Credentials in Public Media          | 5          | Medium   | 24   |
| 12     | Poison Null Byte - File Extension Restriction Bypass         | 4          | Medium   | 25   |
| 13     | Information Disclosure - Admin Email via Product Page        | 3          | Low      | 26   |
| 14     | Information Disclosure - User Emails via Reviews             | 3          | Low      | 27   |
| 15     | Missing Access Control - Unauthorized Deletion of Reviews    | 2          | Low      | 28   |

TEMPLATE NOTE: (Sorting by descending risk score)

# 1 - SQL Injection – Admin Login Bypass

| CRITICAL (10/10)        |        |
|-------------------------|--------|
| Exploitation Likelihood | Likely |
| Business Impact         | Major  |
| Remediation Difficulty  | Easy   |

## Security Implications

An attacker can log into the administrator account without knowing the password by injecting a SQL payload into the login form, granting full administrative access to the application.

## Analysis

The login endpoint does not sanitize user input before including it in the SQL query. By entering ' or 1=1-- as the email and any value as the password, the backend SQL query always returns true and authenticates as the first user in the database (the admin). The admin email admin@juice-sh.op was discovered via the Apple Juice product page.

Flag: 690fa3247a99d651e0b26f947baf0b79b4f404a9

**Request**

| Pretty                                                                                                              | Raw | Hex |
|---------------------------------------------------------------------------------------------------------------------|-----|-----|
| 1 POST /rest/user/login HTTP/1.1                                                                                    |     |     |
| 2 Host: 10.114.140.87                                                                                               |     |     |
| 3 Content-Length: 52                                                                                                |     |     |
| 4 Accept-Language: en-US,en;q=0.9                                                                                   |     |     |
| 5 Accept: application/json, text/plain, */*                                                                         |     |     |
| 6 Content-Type: application/json                                                                                    |     |     |
| 7 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/144.0.0.0 Safari/537.36 |     |     |
| 8 Origin: http://10.114.140.87                                                                                      |     |     |
| 9 Referer: http://10.114.140.87/                                                                                    |     |     |
| 10 Accept-Encoding: gzip, deflate, br                                                                               |     |     |
| 11 Cookie: language=en; cookieconsent_status=dismiss                                                                |     |     |
| 12 Connection: keep-alive                                                                                           |     |     |
| 13                                                                                                                  |     |     |
| 14 {                                                                                                                |     |     |
| "email":"' or 1=1--",                                                                                               |     |     |
| "password":"test-password"                                                                                          |     |     |
| }                                                                                                                   |     |     |

**Figure 1**

## Recommendations

- Use parameterized queries / prepared statements for all database interactions.
- Implement input validation and server-side sanitization on all login fields.
- Deploy a Web Application Firewall to detect SQL injection patterns.

## 2 - SQL Injection – Account Takeover (Bender)

| HIGH RISK (8/10)        |        |
|-------------------------|--------|
| Exploitation Likelihood | Likely |
| Business Impact         | Major  |
| Remediation Difficulty  | Easy   |

### Security Implications

Any registered user account can be hijacked without a password using a SQL comment-based injection on the login page.

### Analysis

By entering `bender@juice-sh.op'--` as the email, the SQL query ignores the password check entirely and authenticates as Bender. The account email was discovered via a Banana Juice product review. Flag: `5ff5052e879e6fef64124e64c82c84ebc809c6c4`

**Request**

Pretty Raw Hex

```
1 POST /rest/user/login HTTP/1.1
2 Host: 10.114.140.87
3 Content-Length: 48
4 Accept-Language: en-US,en;q=0.9
5 Accept: application/json, text/plain, */*
6 Content-Type: application/json
7 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/144.0.0.0 Safari/537.36
8 Origin: http://10.114.140.87
9 Referer: http://10.114.140.87/
10 Accept-Encoding: gzip, deflate, br
11 Cookie: language=en; cookieconsent_status=dismiss; continueCode=1KbV5a7Q65yx3YJp1kNW4RKp9Xzjd58xAvoElgbLeqVmDBMn8roZw2alnJR9
12 Connection: keep-alive
13
14 {
  "email": "bender@juice-sh.op'--",
  "password": "test"
}
```

**Figure 2**

### Recommendations

- Use parameterized queries / prepared statements for all database interactions.
- Enforce server-side input validation on all authentication endpoints.

### 3 - Broken Authentication - Weak Admin Password

| HIGH RISK (8/10)        |        |
|-------------------------|--------|
| Exploitation Likelihood | Likely |
| Business Impact         | Major  |
| Remediation Difficulty  | Easy   |

#### Security Implications

The administrator account uses a trivially guessable password ( admin123 ), allowing unauthorized access via brute force or simple guessing without any bypass techniques.

#### Analysis

After discovering the admin email through the Apple Juice product page, the password admin123 was guessed manually. The application does not enforce password complexity or account lockout policies. Flag: ff4aebffe31b0ffdea9bdd0207a16a3c01ac6c56

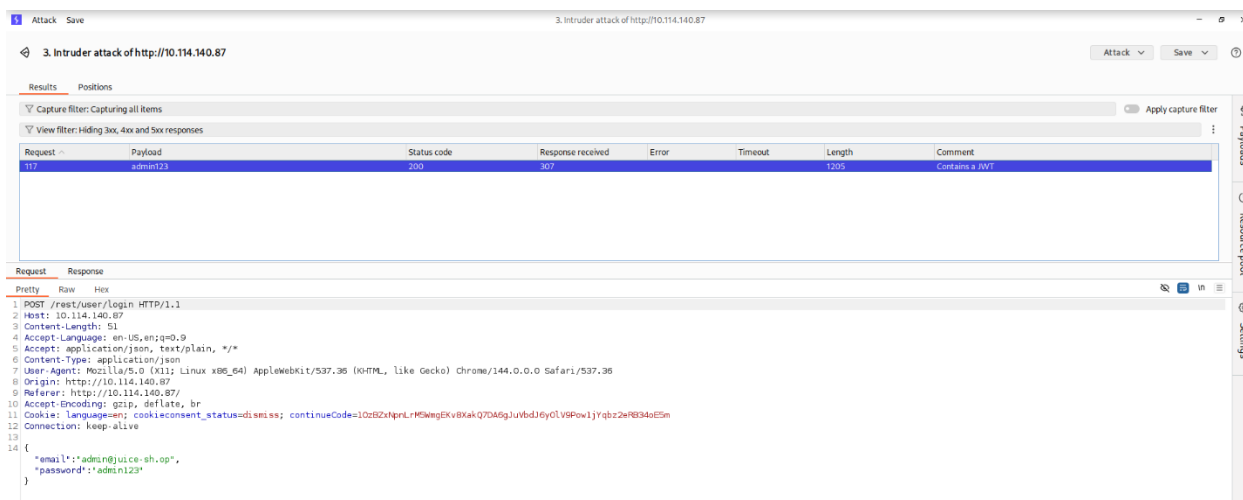


Figure 3

#### Recommendations

- Enforce a strong password policy (minimum 12 characters, mixed case, numbers, symbols).
- Implement account lockout after repeated failed login attempts.
- Enable multi-factor authentication (MFA) for all privileged accounts.

## 4 - Persistent XSS via HTTP Header (True-Client-IP)

| HIGH RISK (8/10)        |          |
|-------------------------|----------|
| Exploitation Likelihood | Likely   |
| Business Impact         | Major    |
| Remediation Difficulty  | Moderate |

### Security Implications

A stored XSS payload injected via the True-Client-IP HTTP header is rendered unsanitized on the Last Login IP page, executing malicious JavaScript for every admin viewing the page.

### Analysis

By intercepting the logout request in Burp Suite and adding the header True-Client-IP: , the payload is stored server-side. Upon next admin login and navigation to the Last Login IP page, the XSS executes. Flag: c37da14686b69a220fd9febd09bb9593e7d0539f



Figure 4

### Recommendations

- Sanitize and validate all HTTP request headers before storing or rendering them.
- Implement a Content Security Policy (CSP) header to restrict script execution sources.
- Encode all output rendered from user-controlled or header-sourced data.

## 5 - Broken Access Control – Admin Panel Exposed via JavaScript

| HIGH RISK (7/10)        |        |
|-------------------------|--------|
| Exploitation Likelihood | Likely |
| Business Impact         | Major  |
| Remediation Difficulty  | Easy   |

### Security Implications

The administration panel route is embedded in client-side JavaScript, allowing any user who inspects the source to navigate to `/#/administration` and access admin functions.

### Analysis

By opening Firefox DevTools > Debugger and loading `main-es2015.js`, the route path: 'administration' was found via text search for "admin". Navigating to `/#/administration` while authenticated as admin revealed full control over user reviews and feedback. Flag: `71aeb3b0bf01cc6e488f0207bb62f79b41454a87`

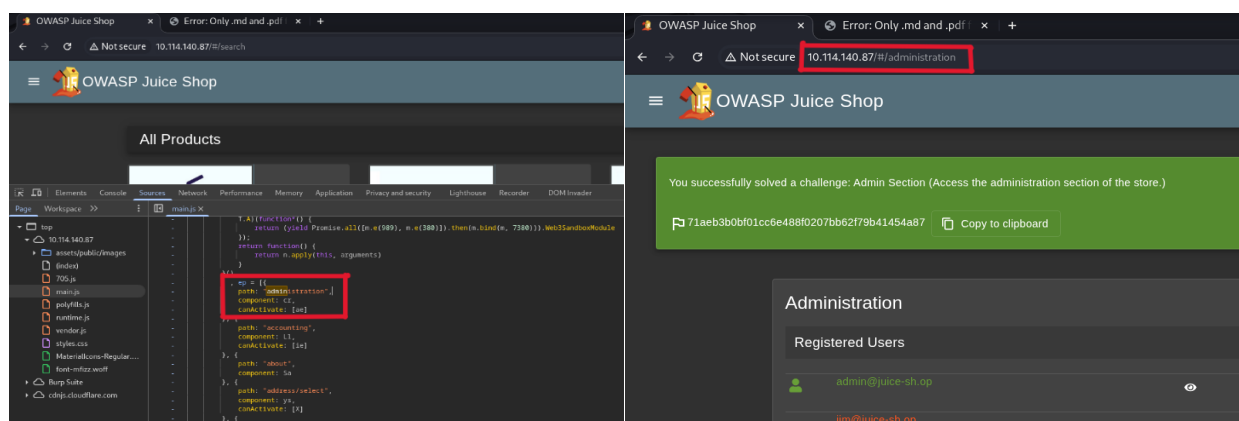


Figure 5&6

### Recommendations

- Enforce server-side authorization checks on all admin API endpoints; do not rely on clientside route hiding.
- Restrict the administration panel to specific IP ranges or require additional authentication.
- Remove or obfuscate sensitive route information from bundled JavaScript files.

## 6 - Reflected XSS via Order Tracking ID

| MEDIUM RISK (6/10)      |          |
|-------------------------|----------|
| Exploitation Likelihood | Likely   |
| Business Impact         | Moderate |
| Remediation Difficulty  | Easy     |

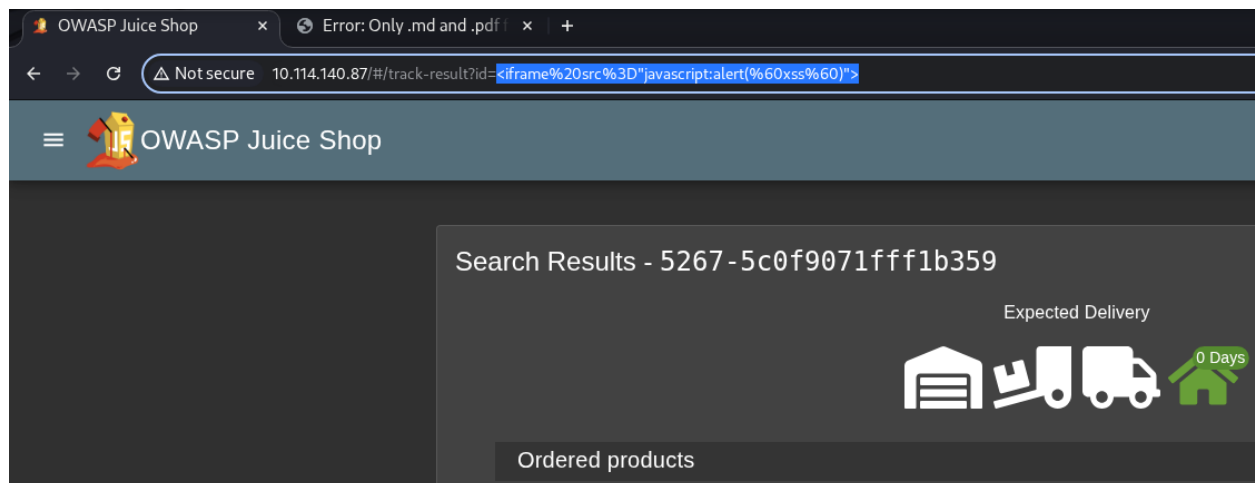
### Security Implications

User-supplied input in the order tracking URL parameter is reflected in the page response without sanitization, enabling reflected XSS attacks against any user who clicks a crafted link.

### Analysis

By navigating to Order History, clicking the truck (tracking) icon, and replacing the tracking ID in the URL with , the XSS payload executes on page load. Flag:

305021787d3e9cd9cebc057a021c2504550bb3b6



**Figure 7**

### Recommendations

- Sanitize and encode all URL parameters before including them in server responses or rendering them in the DOM.
- Implement Content Security Policy (CSP) headers.



## 7 - DOM XSS via Search Bar

| MEDIUM RISK (6/10)      |          |
|-------------------------|----------|
| Exploitation Likelihood | Likely   |
| Business Impact         | Moderate |
| Remediation Difficulty  | Easy     |

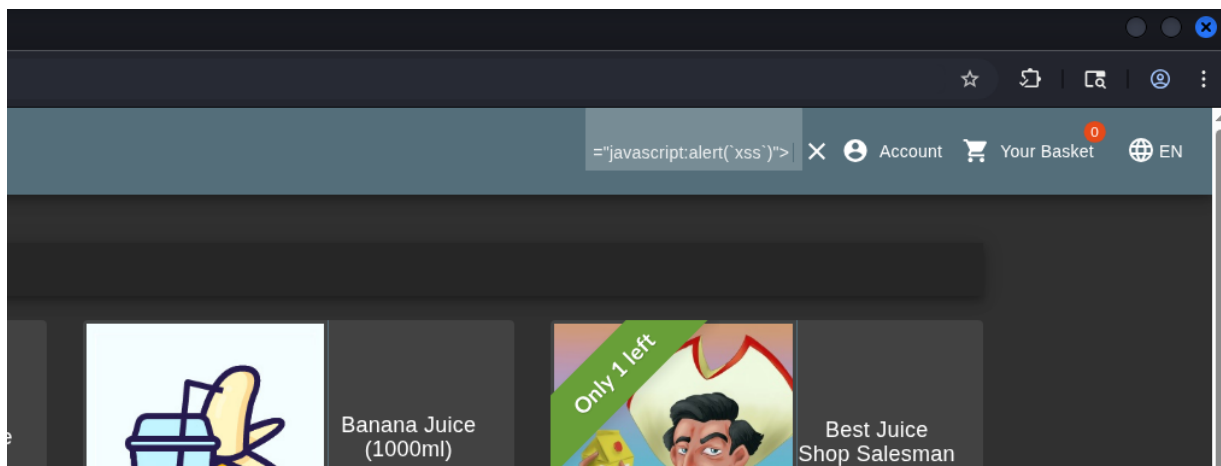
### Security Implications

The application search bar passes user input directly into the DOM without sanitization, allowing DOM-based XSS payloads to execute in the browser.

### Analysis

Entering into the search bar triggered a JavaScript alert dialog. The search parameter is processed client-side and written into the DOM unsafely. Flag:

4a31a4fe0954199566e360a873802bf64d0d0a84



**Figure 8**

### Recommendations

- Avoid using innerHTML or other unsafe DOM manipulation methods with user-supplied data.
- Use safe DOM APIs such as `textContent` or `createTextNode`.
- Implement a strict Content Security Policy (CSP) to prevent inline script execution.

## 8 - Insecure Direct Object Reference - View Other Users' Baskets

| MEDIUM RISK (6/10)      |          |
|-------------------------|----------|
| Exploitation Likelihood | Likely   |
| Business Impact         | Moderate |
| Remediation Difficulty  | Easy     |

### Security Implications

The `/api/basket/{id}` endpoint does not validate that the requesting user owns the basket, allowing any authenticated user to view another user's shopping basket by manipulating the ID.

### Analysis

While logged in as admin, intercepting the GET `/rest/basket/1` request in Burp Suite and changing the basket ID from 1 to 2 returned another user's basket contents without any authorization error. Flag: `e6982b34b6734ceadd28e5019b251f929a80b815`

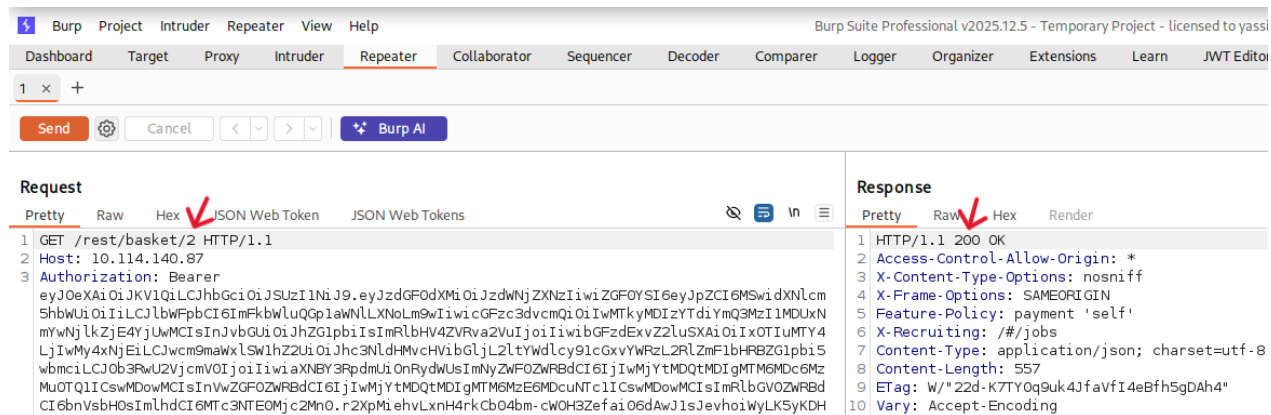


Figure 9

### Recommendations

- Implement server-side ownership checks on all object references.
- Use indirect references (e.g., session-tied tokens) rather than predictable sequential IDs.

## 9 - Sensitive Data Exposure - Public /ftp/ Directory

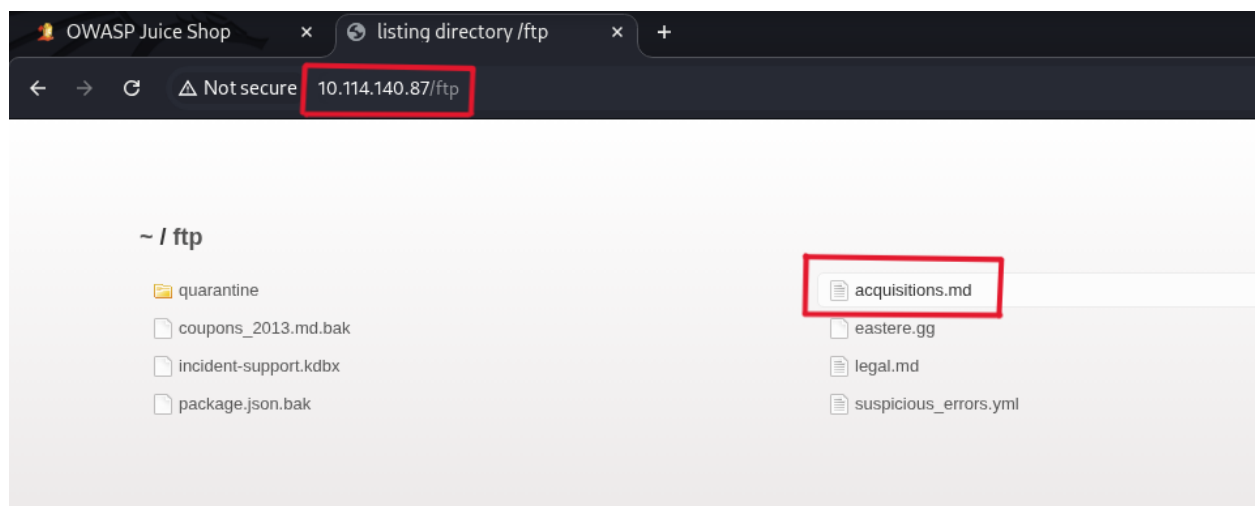
| MEDIUM RISK (6/10)      |        |
|-------------------------|--------|
| Exploitation Likelihood | Likely |
| Business Impact         | Major  |
| Remediation Difficulty  | Easy   |

### Security Implications

The /ftp/ directory is publicly accessible without authentication, exposing confidential business documents including an acquisitions file and developer backup files.

### Analysis

Navigating to <http://10.114.140.87/ftp/> revealed a directory listing containing legal.md , acquisitions.md , and package.json.bak among others. Discovered via the "Check out our terms of use" link on the About Us page. Flag: 8d2072c6b0a455608ca1a293dc0c9579883fc6a5



**Figure 10**

### Recommendations

- Remove the /ftp/ directory from public web server access immediately.
- Implement access controls and authentication for any file-serving directories.
- Audit all web-accessible directories for unintended file exposure.

## 10 - Broken Authentication - Password Reset via Weak Security Question (Jim)

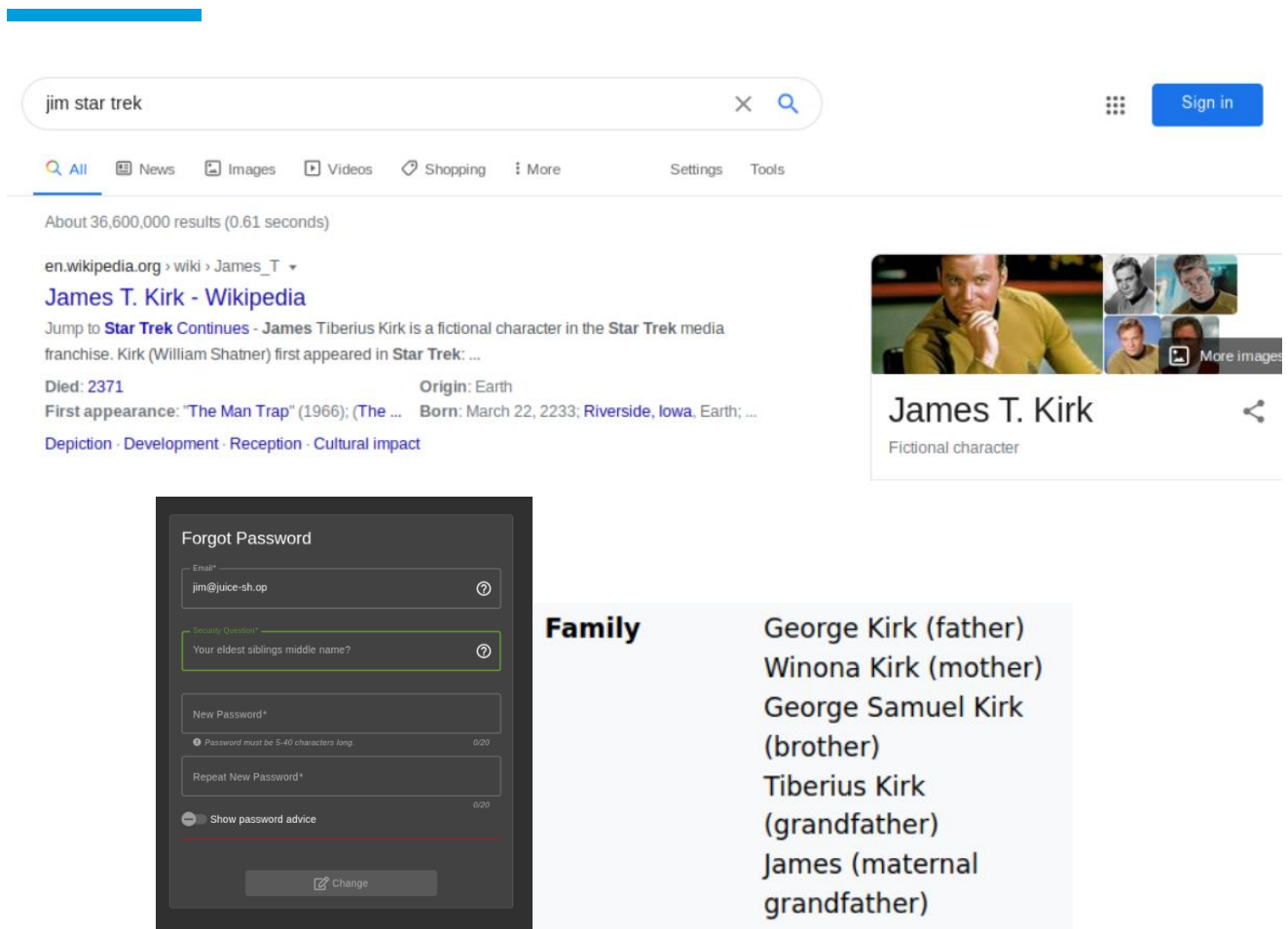
| MEDIUM RISK (5/10)      |        |
|-------------------------|--------|
| Exploitation Likelihood | Likely |
| Business Impact         | Major  |
| Remediation Difficulty  | Easy   |

### Security Implications

Jim's account password can be reset by anyone who can research the answer to his security question, which references a publicly known pop culture fact.

### Analysis

Jim's email ( jim@juice-sh.op ) was found in a Green Smoothie product review mentioning a "replicator" (a Star Trek reference). His security question answer ( Samuel , his brother's middle name) was researchable via OSINT, enabling a full password reset via the Forgot Password page. Flag: 3c3e2d6ef99b733b947e92f8e2a9ed08bf57ea63



**Figure 11,12&13**

## Recommendations

- Replace knowledge-based security questions with multi-factor authentication.
- Implement rate limiting and CAPTCHA on the password reset endpoint.

## 11 – Broken Authentication – Credentials Embedded in Public Media

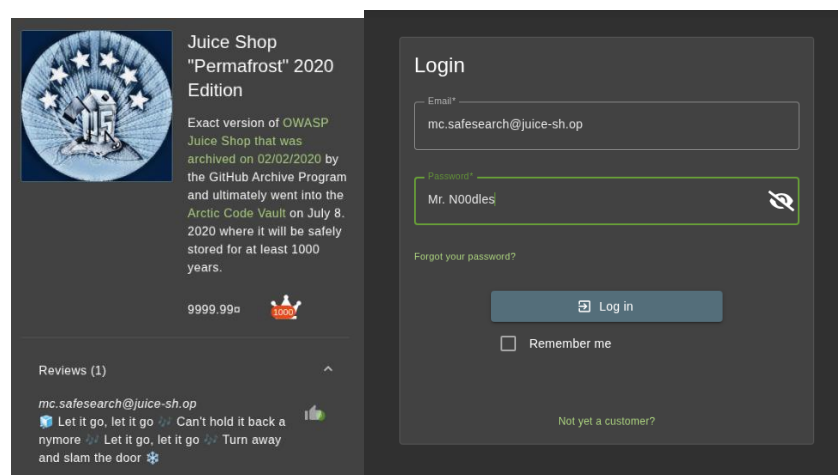
| MEDIUM RISK (5/10)      |          |
|-------------------------|----------|
| Exploitation Likelihood | Likely   |
| Business Impact         | Moderate |
| Remediation Difficulty  | Easy     |

### Security Implications

MC SafeSearch's account password is disclosed in a publicly available promotional video, allowing anyone who watches it to log into the account.

### Analysis

The promotional video (Juice Shop Permafrost 2020 Edition) includes a segment where the character states his password is "Mr. Noodles" with vowels replaced by zeros, yielding Mr. N00dles for account mc.safesearch@juice-sh.op . Flag: bb105418e73708ceccf1a7b2491f434b8f5230e4



**Figure 14&15**

### Recommendations

- Never embed credentials or password hints in marketing or public-facing media..
- Enforce periodic password changes and ensure credentials are not shared externally.

## 12 - Poison Null Byte - File Extension Restriction Bypass

| MEDIUM RISK (4/10)      |          |
|-------------------------|----------|
| Exploitation Likelihood | Possible |
| Business Impact         | Moderate |
| Remediation Difficulty  | Moderate |

### Security Implications

The /ftp/ directory restricts file downloads to .md and .pdf extensions, but this control can be bypassed using a URL-encoded Poison Null Byte, allowing download of arbitrary files.

### Analysis

Attempting to download package.json.bak returned a 403 error. By appending %2500.md to the filename in the URL (URL-encoded null byte %00 ), the server was tricked into allowing the download of the restricted backup file. Flags: dad737329153068527159005db4eb139e86e76ba | cfdeea14e8f01b4952722fd0e4a77f1928593c9a

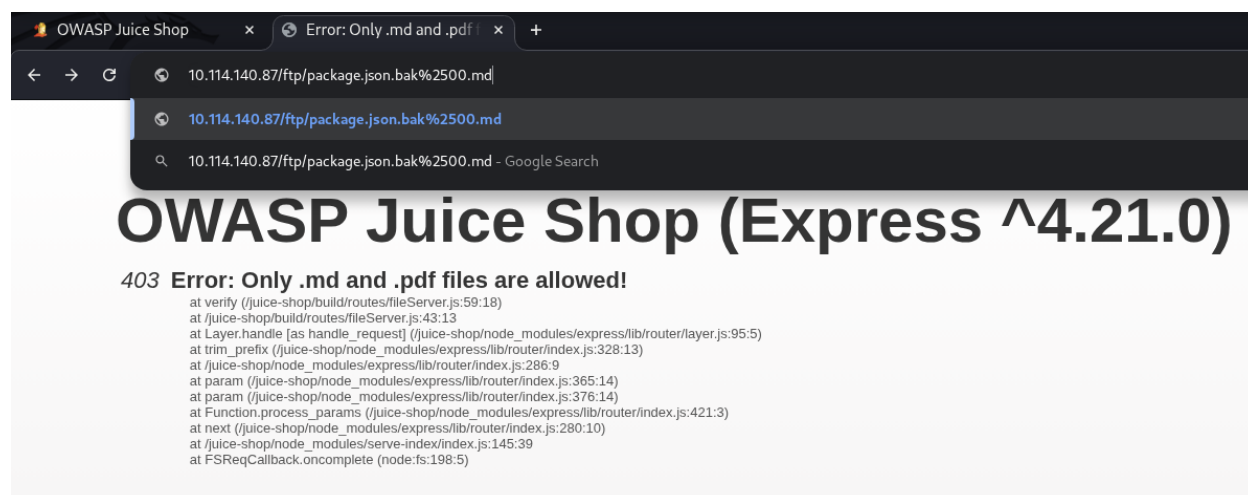


Figure 16

### Recommendations

- Implement server-side file type validation using MIME type detection, not filename strings.
- Sanitize and reject URLs containing null bytes or other special characters before processing.
- Remove the /ftp/ directory from public access entirely.

## 13 - Information Disclosure - Admin Email via Product Page

| LOW RISK (3/10)         |        |
|-------------------------|--------|
| Exploitation Likelihood | Likely |
| Business Impact         | Minor  |
| Remediation Difficulty  | Easy   |

### Security Implications

The administrator's email address is displayed publicly on a product detail page, providing attackers with a valid username for brute force or social engineering attacks.

### Analysis

Clicking on the Apple Juice product page revealed the email `admin@juice-sh.op` in the review section without requiring authentication.

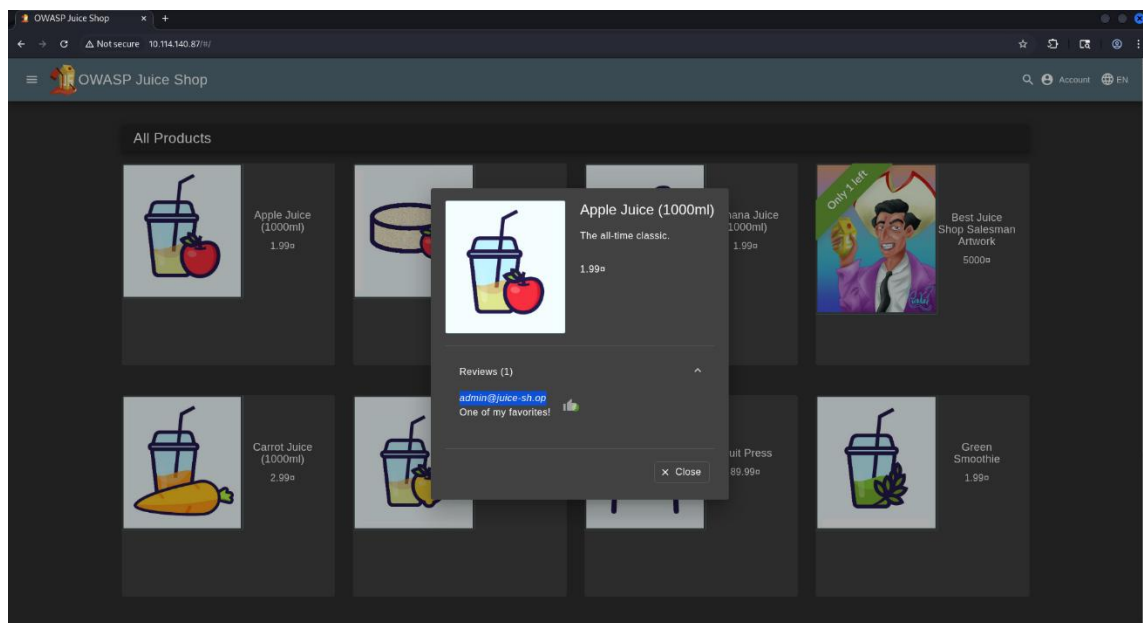


Figure 17

### Recommendations

- Avoid displaying internal or administrative email addresses on public-facing pages.
- Use display names or obfuscated identifiers for admin accounts.



## 14 - Information Disclosure - User Emails via Product Reviews

| LOW RISK (3/10)         |        |
|-------------------------|--------|
| Exploitation Likelihood | Likely |
| Business Impact         | Minor  |
| Remediation Difficulty  | Easy   |

### Security Implications

Multiple user email addresses are exposed publicly in product reviews, enabling targeted phishing, account enumeration, and credential stuffing attacks.

### Analysis

Product reviews on Green Smoothie ( jim@juice-sh.op ), Banana Juice ( bender@juice-sh.op ), and the Permafrost 2020 Edition page ( mc.safesearch@juice-sh.op ) displayed full email addresses without authentication.

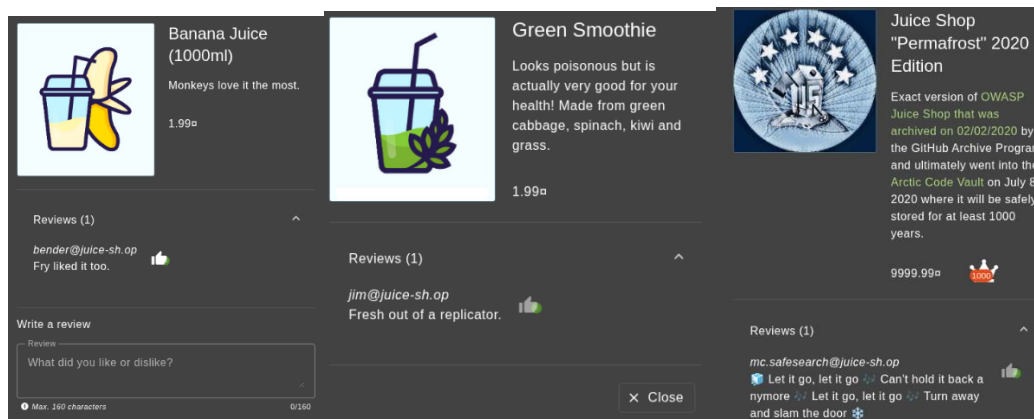


Figure 18,19&20

### Recommendations

- Replace email addresses with display names or usernames in publicly visible reviews.
- Audit all public-facing pages for unintended user data exposure.

## 15 – Missing Access Control – Unauthorized Deletion of Customer Reviews

| LOW RISK (2/10)         |          |
|-------------------------|----------|
| Exploitation Likelihood | Possible |
| Business Impact         | Minor    |
| Remediation Difficulty  | Easy     |

### Security Implications

The admin panel allows deletion of all customer reviews including 5-star feedback without any audit trail, which could be abused to manipulate reputation or suppress legitimate user feedback.

### Analysis

While authenticated as admin, navigating to `/#/administration` displayed all customer reviews with a delete (bin) icon. Clicking the icon next to any 5-star review permanently deleted it with no confirmation or logging. Flag: `78231b75c0b2180b7e964dcbb1ab3c3f58639f2e`

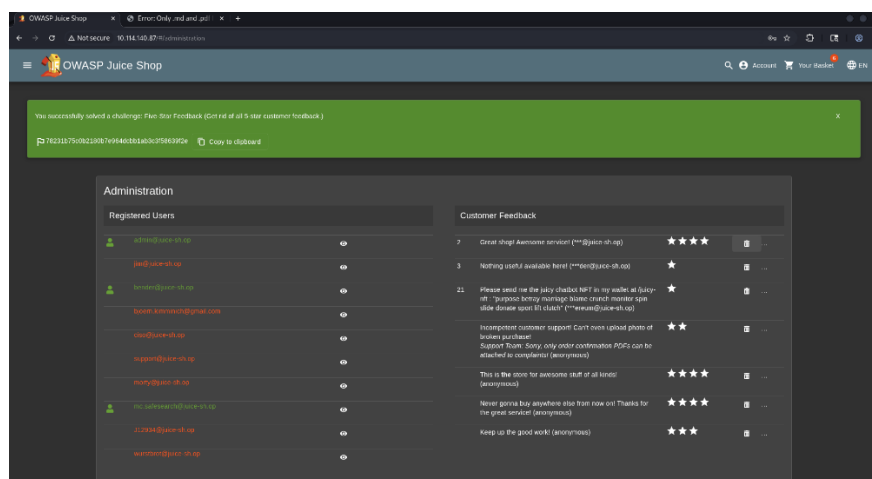


Figure 10

### Recommendations

- Implement an audit log for all administrative actions including content deletion.
- Add a confirmation step and require a reason before deleting user-generated content.
-

## TOOLS USED

| TOOL                               | DESCRIPTION                                                |
|------------------------------------|------------------------------------------------------------|
| <b>BurpSuite Community Edition</b> | HTTP proxy for intercepting, modifying, replaying requests |
| <b>Web Browser</b>                 | Manual application walking, reconnaissance                 |
| <b>Firefox DevTools (Debugger)</b> | Inspect bundled JS files, discover hidden route            |

**Table A.1:** Tools used during assessment

# ENGAGEMENT INFORMATION

## Client Information

|                        |                                                                                                                                                                                                                                                     |
|------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Client</b>          | OWASP Juice Shop                                                                                                                                                                                                                                    |
| <b>Primary Contact</b> | Yassin Bassam,<br>Penetration Tester                                                                                                                                                                                                                |
| <b>Approvers</b>       | The following people are authorized to change the scope of engagement and modify the terms of the engagement <ul style="list-style-type: none"><li>• Mahmoud Ahmed</li><li>• Naira Elsayed</li><li>• Habiba Khamis</li><li>• Mohamed Wael</li></ul> |

## Version Information

| Version | Date       | Description              |
|---------|------------|--------------------------|
| 1.0     | April 2026 | Initial report to client |

## Contact Information

|              |                            |
|--------------|----------------------------|
| <b>Name</b>  | Team 4 Consulting          |
| <b>Email</b> | yassinbassam2005@gmail.com |